Random Forest

Aprendizaje Supervisado · Bootstrap, Variables Aleatorias y Votación · Convocatoria a la Selección

Inteligencia Artificial · Sexto Semestre · Ingeniería de Sistemas de Información · PUCE

Objetivos de Aprendizaje

Al finalizar esta clase serás capaz de:

1. Explicar por qué un árbol de decisión individual es inestable y cómo un ensamble de árboles lo corrige.
2. Describir los tres mecanismos de Random Forest: Bootstrap, variables aleatorias en cada nodo y votación.
3. Entrenar un RandomForestClassifier en un Pipeline, compararlo con un árbol e interpretar el OOB (Out-of-Bag) y las importancias.
4. Reentrenar el modelo final con el 100% de los datos, exportarlo con joblib y usarlo con datos nuevos.

El Problema: un Solo Árbol es Inestable

Lo que vimos en la clase de Árboles de Decisión

Inestabilidad (dataset de crédito)

- Con el 80% de los datos la raíz era: deuda_actual ≤ 5 975.
- Con el 100% la raíz pasó a: deuda_actual ≤ 8 014.
- Pocos datos distintos → otro árbol, otras reglas.

Sobreajuste (dataset de hoy)

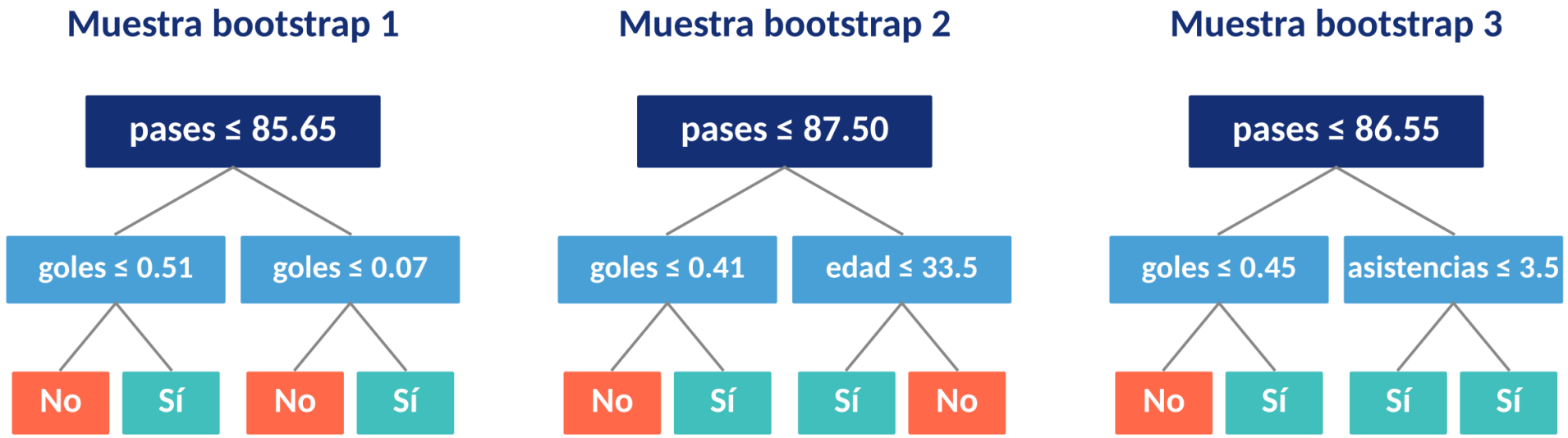
- Árbol sin límite: 91 hojas y 17 niveles.
- Accuracy en entrenamiento: 1.000.
- Accuracy en prueba: 0.778 → memorizó el ruido.

Varianza: cuánto cambia un modelo cuando cambian un poco los datos de entrenamiento. Un árbol sin límite tiene varianza alta.

Idea: entrenar muchos árboles distintos y combinar sus respuestas

Tres Muestras, Tres Árboles Distintos

Árboles reales de profundidad 2 entrenados con tres muestras de jugadores_liga_pro.csv



Hojas: «Sí» = convocado · «No» = no convocado (pases = pases_completados_pct · goles = goles_por_partido)

- Las tres raíces usan pases_completados_pct, pero con umbrales distintos: 85.65, 87.50 y 86.55.
- En el segundo nivel aparecen variables distintas: goles, edad y asistencias.
- Cada árbol "ve" el problema de forma diferente: esa diversidad es la materia prima del bosque.
- Una división puede terminar en dos hojas «Sí»: cambia la proporción de convocados, no la decisión.

¿Qué es Random Forest?

Un ensamble: muchos modelos que deciden juntos

Definición: método de aprendizaje por ensamble (ensemble learning: combinar varios modelos en uno) que entrena muchos árboles de decisión distintos y combina sus predicciones por votación (clasificación) o promedio (regresión).

En lenguaje simple

- En vez de 1 árbol se entrenan 100 (n_estimators).
- Cada árbol es distinto a propósito.
- Gana la clase que eligen más árboles.

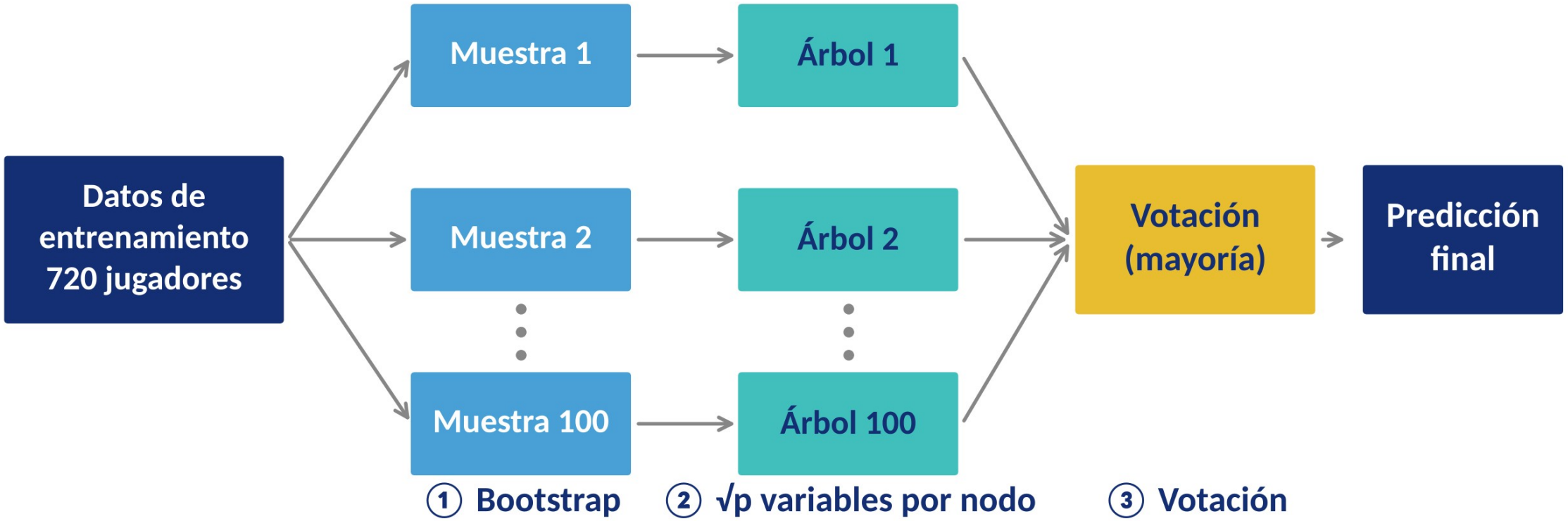
Analogía: un cuerpo técnico

- Un solo observador puede equivocarse por el último partido que vio.
- 100 observadores que vieron partidos distintos se equivocan menos al votar.

Fuente: Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32.

Cómo se Construye un Bosque

Los tres mecanismos de Random Forest de un vistazo



Datos distintos + variables distintas → árboles diversos → votación → predicción estable

Mecanismo 1 — Bootstrap: Datos Distintos para cada Árbol

Ejemplo ilustrativo con 10 jugadores (J1 … J10)

Bootstrap: muestreo CON reemplazo del mismo tamaño que el original: un jugador puede salir varias veces.



Cómo leerlo

- Dorado: jugador repetido.
- Celeste: aparece una vez.
- Fuera (OOB = Out-of-Bag): no fue elegido para ese árbol.
- Parámetro: bootstrap=True (defecto).

¿Cuántos Jugadores Distintos Recibe cada Árbol?

Cada fila tiene probabilidad 1/n de ser elegida en cada uno de los n sorteos

Probabilidad de que un jugador NO salga en la muestra

$$P(\text{no elegido}) = \left(1 - \frac{1}{n}\right)^{n} \approx e^{-1} \approx 0.368$$

Con los 720 jugadores de entrenamiento

$$n = 720: \quad \left(1 - \frac{1}{720}\right)^{720} = 0.368 \Rightarrow 63.2\% \text{ distintos}$$

Comprobado con los datos

- Tres muestras reales: 63.2%, 62.6% y 62.6% de jugadores distintos.

Lo que queda fuera

- ≈ 36.8% de jugadores no ven ese árbol: muestras Out-of-Bag (OOB).

Mecanismo 2 — Variables Aleatorias en cada División

Además de datos distintos, cada nodo mira solo algunas variables

Árbol de decisión clásico

- En cada nodo prueba TODAS las variables.
- Si se entrenan muchos árboles así, todos eligen casi la misma raíz (diapositiva 4: las tres usan pases).

Random Forest

- En cada nodo sortea solo m variables y elige la mejor entre ellas.
- Parámetro: max_features='sqrt' (defecto).

Con p = 22 columnas (tras One-Hot)

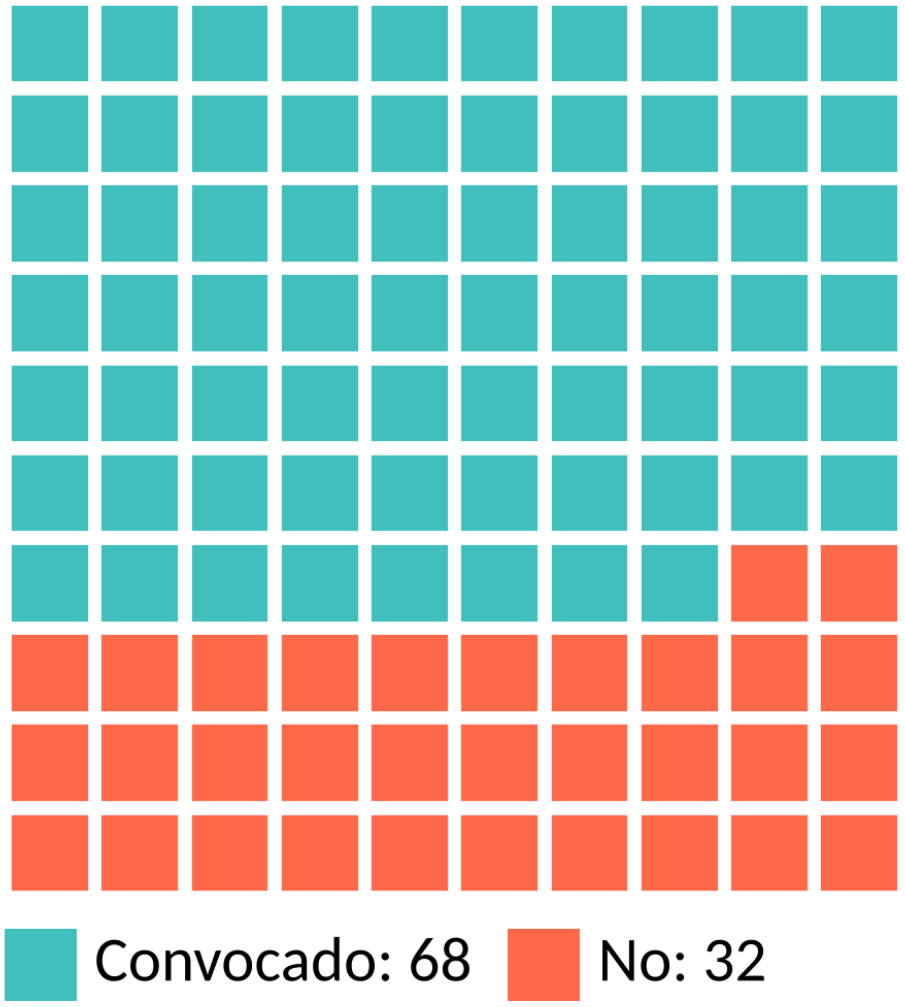
$$m = \lfloor \sqrt{p} \rfloor = \lfloor \sqrt{22} \rfloor = \lfloor 4.69 \rfloor = 4$$

⚠️ **Importante:** el sorteo se repite en CADA nodo, no una sola vez por árbol.

Así variables menos dominantes también tienen oportunidad de usarse

Mecanismo 3 — Votación: la Decisión del Bosque

Votos reales de los 100 árboles para un jugador del conjunto de prueba (JUG0080)



El jugador

- Defensa de Universidad Católica, 25 años.
- 89.9% de pases completados, 10 partidos, 0 tarjetas.
- En la realidad sí fue convocado: el bosque acierta.

$$\hat{y}(x) = \text{mayoría}\ \{\, h_1(x),\ h_2(x),\ \ldots,\ h_B(x) \,\}$$

68 votos Convocado vs. 32 No → CONVOCADO

predict_proba() en Random Forest

La probabilidad que entrega el bosque para cada clase

Promedio de las probabilidades de los B árboles

$$\hat{p}(x) = \frac{1}{B}\sum_{b=1}^{B} p_b(x)$$

Lectura

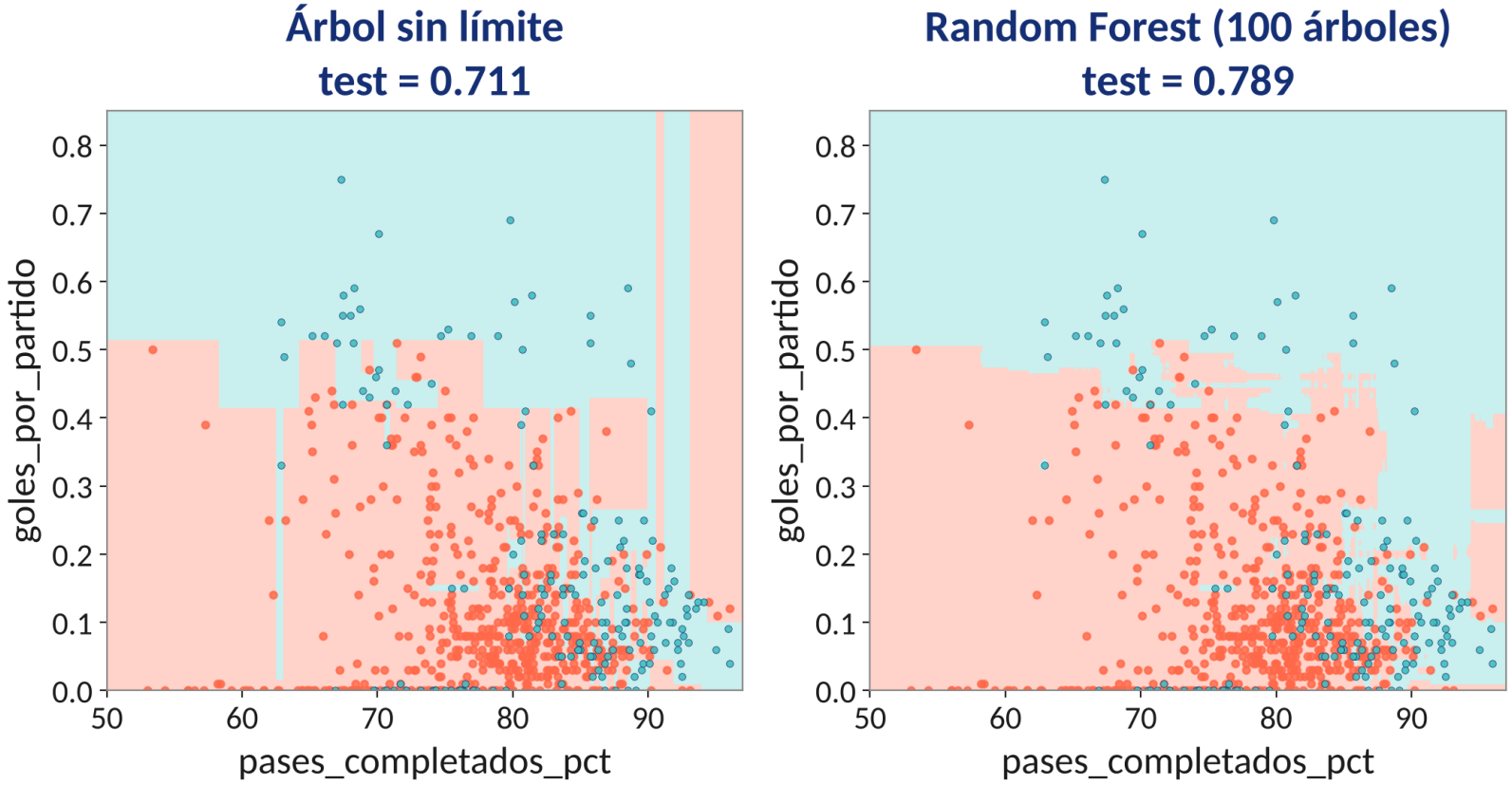
- p_b(x): proporción de cada clase en la hoja del árbol b.
- Rango 0 a 1 por clase; las clases suman 1.

⚠️ **Importante:** scikit-learn NO cuenta votos: promedia probabilidades. Con árboles sin límite (defecto) cada hoja es pura y el resultado coincide con la proporción de votos.

Jugador JUG0080: `predict_proba() → [0.32, 0.68]` = 32 y 68 votos de 100 árboles.

¿Por qué Funciona? Fronteras de Decisión

Solo 2 variables, para poder dibujarlo: celeste = convocado · coral = no convocado



Promediar árboles diversos reduce las islas de un solo punto: menor varianza

Out-of-Bag (OOB): una Validación Incluida

Usa los jugadores que cada árbol NO vio durante su entrenamiento

Entrenamiento: 720 jugadores

≈ 455 únicos (63.2%) entrenan el árbol 1

≈ 265 OOB (36.8%) lo evalúan

Cada jugador es OOB en ≈ 37 de los 100 árboles: esos árboles votan su predicción → oob_score_

¿Por qué es útil?

- Es una evaluación honesta: cada voto viene de árboles que no vieron a ese jugador.
- No necesita separar datos extra.
- Activar: oob_score=True.
- Rango 0 a 1 → 1 = deseado.

OOB = 0.826 · prueba = 0.878: la diferencia es azar (prueba usa solo 180 jugadores; OOB usa los 720)

Importancia de Variables: feature_importances_

Reducción de Gini de cada variable, promediada en los 100 árboles (top 10 real)



Lectura

- pases_completados_pct (0.227) y goles_por_partido (0.174) son las más usadas.
- Dorado: columnas One-Hot de posicion.
- Rango 0 a 1; todas suman 1.
- Indica qué usó el modelo, no causa y efecto.

La Clase RandomForestClassifier()

X_train debe ser numérico: en este curso el bosque se entrena dentro de un Pipeline (más adelante)

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(n_estimators=100,
                            oob_score=True,
                            random_state=42)
rf.fit(X_train, y_train)      # entrena los 100 árboles
y_pred = rf.predict(X_test)   # votación de los árboles
```

Atributo: existe solo después de fit()	Qué devuelve
`estimators_`	Lista con los 100 árboles entrenados
`oob_score_`	Accuracy OOB (requiere oob_score=True)
`feature_importances_`	Importancia de cada variable (suman 1)
`classes_`	Clases aprendidas: [0, 1]

Parámetros del Constructor

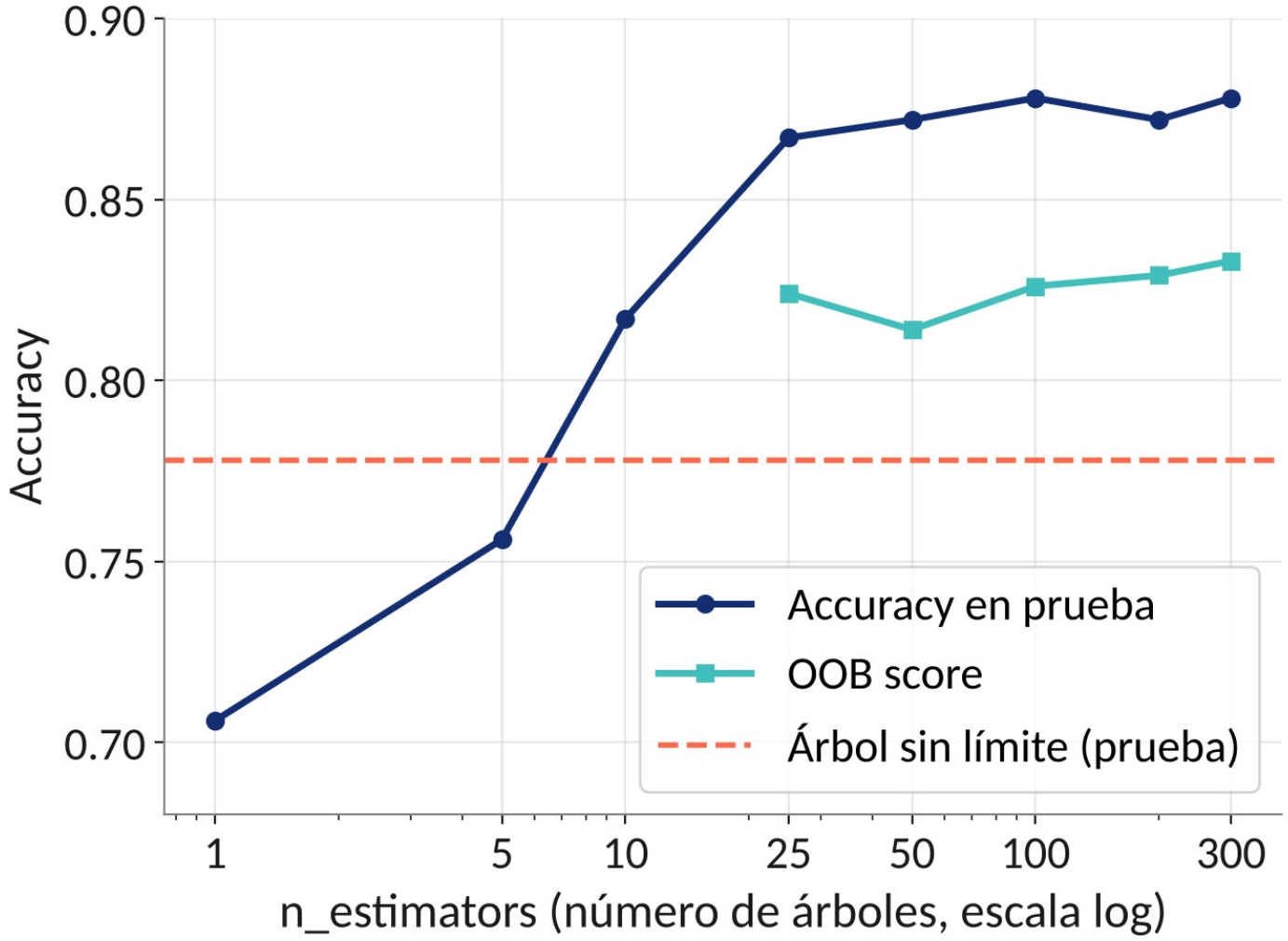
Se pasan al crear el modelo, antes de fit()

Parámetro	Defecto	Qué controla
`n_estimators`	100	Número de árboles del bosque
`max_features`	'sqrt'	Variables sorteadas en cada nodo (√p)
`bootstrap`	True	Muestreo con reemplazo para cada árbol
`max_depth`	None	Profundidad máxima (None = sin límite)
`min_samples_leaf`	1	Casos mínimos en cada hoja
`oob_score`	False	Calcular el OOB score
`n_jobs`	None	Núcleos del procesador (None = 1; -1 = todos)
`random_state`	None	Semilla para resultados repetibles

Los valores por defecto suelen funcionar bien: el bosque necesita poco ajuste

Curva Real: Número de Árboles vs. Accuracy

Accuracy: rango 0 a 1 → 1 = deseado



Lectura

- 1 árbol: 0.706.
- 10 árboles: 0.817.
- 100 árboles: 0.878.
- 300 árboles: 0.878.
- Agregar árboles no aumenta el sobreajuste: el accuracy se estabiliza.
- OOB desde 25 árboles: con menos, algunos jugadores nunca quedan fuera y no se puede calcular.

¿Hay que Escalar o Codificar?

El bosque está hecho de árboles: hereda sus reglas

Escalado: no es necesario

- Cada nodo compara con un umbral: pases ≤ 85.65.
- Escalar no cambia el orden de los casos.

Codificación: sí es necesaria

- equipo y posicion son texto.
- Se usa OneHotEncoder dentro de un ColumnTransformer.

⚠️ **No usar LabelEncoder en X:** convierte equipos en 0, 1, 2… e inventa un orden que no existe (Aucas < Emelec). LabelEncoder está pensado para la variable objetivo y.

Pipeline = ColumnTransformer (codificar) + RandomForestClassifier

Dataset del Taller — Convocatoria a la Selección

jugadores_liga_pro.csv · 900 jugadores de la LigaPro · sintético · sin nulos · convocados: 225 (25%)

Variable	Tipo	Descripción
`equipo, posicion`	Texto	Club (10) y posición (4)
`edad`	Numérica	Años (18 a 37)
`partidos_jugados`	Numérica	Partidos en la temporada (2 a 34)
`minutos_jugados`	Numérica	Minutos totales en la temporada
`goles_por_partido`	Numérica	Promedio de goles por partido
`asistencias`	Numérica	Pases de gol en la temporada
`pases_completados_pct`	Numérica	% de pases completados
`tarjetas_amarillas`	Numérica	Tarjetas amarillas en la temporada
`tarjetas_rojas`	Numérica	Tarjetas rojas en la temporada
`convocado`	Objetivo (y)	1 = convocado · 0 = no

Clases desbalanceadas (75/25): además de Accuracy se mira Recall y F1 (equilibrio entre Precision y Recall) de Convocado.

Flujo Completo (1/5): Datos y División

Construir X, y y separar el 20% para prueba

```
import pandas as pd
from sklearn.model_selection import train_test_split

df = pd.read_csv('jugadores_liga_pro.csv')
X = df.drop(columns=['jugador_id', 'convocado'])
y = df['convocado']                          # 0 o 1

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
```

- jugador_id se elimina: es un identificador, no ayuda a predecir.
- stratify=y mantiene el 25% de convocados en entrenamiento (720) y prueba (180).

Flujo Completo (2/5): Codificar el Texto

Misma idea del ColumnTransformer de Regresión Logística, sin escalador

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline

cols_texto = ['equipo', 'posicion']
preproc = ColumnTransformer(
    [('cat', OneHotEncoder(handle_unknown='ignore'),
      cols_texto)],
    remainder='passthrough',           # numéricas igual
    verbose_feature_names_out=False)  # nombres cortos
```

- remainder='passthrough': las 8 columnas numéricas pasan sin cambios.
- Resultado: 10 equipos + 4 posiciones + 8 numéricas = 22 columnas.

Flujo Completo (3/5): Árbol y Bosque en un Pipeline

El mismo objeto preproc se usa en ambos: los dos modelos reciben las mismas 22 columnas

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

arbol = Pipeline([('preproc', preproc),
                  ('modelo', DecisionTreeClassifier(
                      random_state=42))])
bosque = Pipeline([('preproc', preproc),
                   ('modelo', RandomForestClassifier(
                       n_estimators=100, oob_score=True,
                       random_state=42))])
arbol.fit(X_train, y_train)    # árbol sin límite
bosque.fit(X_train, y_train)   # 100 árboles
```

⚠️ **Pipeline no copia preproc:** cada fit() vuelve a ajustar el mismo objeto. Aquí no cambia nada: ambos usan X_train.

Flujo Completo (4/5): Evaluar en Prueba

score() devuelve el Accuracy · classification_report() da Precision, Recall y F1 por clase

```
from sklearn.metrics import classification_report

print('Árbol  train:', arbol.score(X_train, y_train))
print('Árbol  test: ', arbol.score(X_test, y_test))
print('Bosque test: ', bosque.score(X_test, y_test))

y_pred = bosque.predict(X_test)
print(classification_report(y_test, y_pred))
```

Métricas (rango 0 a 1 → 1 = deseado): Precision = de los que predice como convocados, cuántos lo son · Recall = convocados reales que detecta · F1 = equilibrio entre ambas.

Resultados Reales: Árbol vs. Bosque

Conjunto de prueba: 180 jugadores, 45 convocados · rango 0 a 1 → 1 = deseado

Modelo	Acc. train	Acc. test	Precision	Recall	F1 (Conv.)
Árbol sin límite	1.000	0.778	0.549	0.622	0.583
Árbol max_depth=3	0.832	0.806	0.647	0.489	0.557
Random Forest (100)	**1.000**	**0.878**	**0.897**	**0.578**	**0.703**

Lectura

- Accuracy: 0.778 → 0.878. F1: 0.583 → 0.703.
- Precision sube (0.549 → 0.897) y Recall baja (0.622 → 0.578): el bosque es más prudente.
- Limitar el árbol (depth 3) ayuda poco.

⚠️ **Ojo:** el bosque también da 1.000 en entrenamiento. Eso es normal: sus árboles crecen sin límite. Se juzga con test y OOB, no con train.

Flujo Completo (5/5): OOB e Importancia de Variables

named_steps[...] recupera cada paso del Pipeline ya entrenado

```
rf = bosque.named_steps['modelo']      # el bosque
pre = bosque.named_steps['preproc']    # el codificador
print('OOB:', rf.oob_score_)           # 0.826

imp = pd.Series(rf.feature_importances_,
                index=pre.get_feature_names_out())
print(imp.sort_values(ascending=False).head(10))
```

- get_feature_names_out(): nombres de las 22 columnas (ej. posicion_Defensa).
- Resultado: el gráfico de importancias de la diapositiva 14.

Ventajas y Limitaciones

Cuándo conviene un bosque y qué cuidar

✔ Ventajas

- Mayor accuracy y más estable que un árbol.
- Agregar árboles no aumenta el sobreajuste.
- No requiere escalar variables.
- Trae OOB e importancias incluidos.
- Robusto: si un árbol se equivoca, los demás lo compensan.

✖ Limitaciones

- No se lee como reglas SI … ENTONCES.
- Más lento y pesado: guarda 100 árboles.
- Las importancias pueden favorecer variables numéricas con muchos valores.
- El archivo .pkl exportado es más grande.

Árbol: cuando hay que explicar la regla · Bosque: cuando importa la exactitud

Entrenamiento Final con el 100% de los Datos

Una vez evaluado, el modelo final se entrena con los 900 jugadores

Modelo de evaluación (80%)

- bosque: sirvió para medir.

Modelo final (100%)

- modelo_final: se exporta.

```
modelo_final = Pipeline([('preproc', preproc),
                         ('modelo', RandomForestClassifier(
                             n_estimators=100,
                             random_state=42))])
modelo_final.fit(X, y)    # X, y completos: no X_train
```

Observa: sin X_test ni OOB que medir, el desempeño esperado es el que ya se estimó con el 80%. Además, fit() vuelve a ajustar el mismo preproc con el 100%: las categorías son las mismas, así que no cambia nada.

Exportación con joblib (1/2)

Un solo archivo .pkl guarda el codificador y los 100 árboles

```
import joblib
joblib.dump(modelo_final, 'modelo_convocatoria_rf.pkl')

# en otro programa (ej. un servicio web):
import joblib
import pandas as pd
modelo = joblib.load('modelo_convocatoria_rf.pkl')
```

Qué se guarda

- El Pipeline completo: OneHotEncoder + bosque.

Qué NO hace falta

- Codificar a mano ni escalar: el Pipeline lo hace.

⚠️ **Versión:** carga el .pkl con la misma versión de scikit-learn con la que se guardó.

Uso con Datos Nuevos (2/2)

Mismas 10 columnas de X, con los mismos nombres y el texto sin codificar

```
nuevos = pd.DataFrame({
    'equipo': ['Emelec', 'Orense', 'Aucas'],
    'posicion': ['Delantero', 'Defensa', 'Mediocampista'],
    'edad': [24, 33, 27],
    'partidos_jugados': [30, 12, 28],
    'minutos_jugados': [2500, 700, 2300],
    'goles_por_partido': [0.55, 0.02, 0.15],
    'asistencias': [6, 0, 7],
    'pases_completados_pct': [78.0, 74.0, 88.0],
    'tarjetas_amarillas': [2, 5, 3],
    'tarjetas_rojas': [0, 1, 0]})
print(modelo.predict(nuevos))                  # [1 0 1]
print(modelo.predict_proba(nuevos)[:, 1])   # prob. convocado
```

Delantero → Convocado (0.73) · Defensa → No (0.02) · Mediocampista → Convocado (0.82)